REPAIR_LEDGER — Atlas Registry v0.1

Milestone

This is the first Atlas Registry build. Where every prior move added a link to
the runtime chain, Atlas Registry v0.1 steps to the side and builds the *dictionary*:
a read-only, canonical registry of the vocabulary Project Witness already uses —
components and versions, workflow types, action types, the status vocabulary, claim
predicates, and case record types. Its purpose is to give later components one
canonical source of allowed terms so they can stop hard-coding vocabulary. It runs no
workflows, calls no components, and decides no truth.

Design decisions

- **One source of truth, built in code.** `registry_model.py` constructs the entire registry dict in Python, and

`registry/atlas_registry.json` is generated from it as
canonical JSON. The same builder backs selfcheck, so the shipped file and the in-code model can never silently drift.

- **The registry records, it does not invent.** Every component, status, predicate,
and record type mirrors what the frozen stack actually produced across Moves 1-20
(e.g. Sentinel's `PASS_STRICT`, ODIN's `CONSISTENT` / `CONTRADICTION_DETECTED` / `INVALID_INPUT`, Case Bundle's `BUNDLED_VERIFIED_CASE`, Companion's `COMPANION_COMPLETED`). Atlas describes the vocabulary; it does not extend it.

- **Truth refusal is structural, not cosmetic.** The verifier refuses, with `FAILED_TRUTH_CLAIM`, any status or predicate whose `truth_claim` is `true`, any truth-like status name (`PROVEN_TRUE`, `AUTHENTIC`, `SOURCE_TRUSTED`, …), and any truth-like predicate (`contents.true`, `file.authentic`, `reality.proven`, …). Every recorded status and predicate carries `truth_claim: false`. The roadmap boundary is enforced too: `WRITE_FILE` / `DELETE_FILE` / `SHELL_COMMAND` / `NETWORK_CALL` action types are refused.

- **Duplicates are scoped correctly.** Component ids, workflow types, action types, and predicates must be globally unique; statuses must be unique *within a status group* (so the same status name may legitimately appear in two different groups).
- **Read-only with two-layer export integrity.** Export writes a byte-identical canonical registry copy, a summary with actual counts, and strict checksums, then self-verifies. `verify-export` recomputes the hashes, re-verifies the registry copy as a valid truth-free registry, and confirms the summary counts match — so a tamper hidden by resyncing the checksums is still caught by the content re-verification.
- **Authoritative freeze gate = explicit command list** (`RELEASE_COMMANDS.md`, including `atlas.py verify --registry`) plus the static `verify_release_manifest.py` (imports only ast/os/sys, no spawn call sites, AST-checked by test 42). No subprocess orchestrator is in the gate; the whole suite runs in process.

Repairs

This was a clean first build of Atlas Registry v0.1: no behavioural self-fixes were
required. The registry generated, round-tripped as canonical JSON, and verified to
`REGISTRY_VERIFIED` on the first run; export and verify-export passed first time. The
only initial failures were the expected documentation-dependent tests (README
disclaimers, the RELEASE_COMMANDS list, and the manifest-verifier presence check),
which cleared once the documentation was written.

Honesty boundary

Atlas records vocabulary and verifies the registry is structurally valid and
truth-free. It never asserts that any evidence is true, any claim is true, any file
is authentic, or any source is trusted — and it refuses to record a registry that
would. The export checksums give tamper-evidence within an export folder, not
tamper-proofness, and not production evidence custody.